

CLEAN CODE

NAME: NOORA NAJEEB ALTAMIMI

What Is Clean Code?

Clean code is code that is easy to read, understand, maintain, and modify. It is written in a clear and organized way to help developers work more efficiently and reduce errors.

Clean code focuses on readability, simplicity, and maintainability, making software easier to develop and improve over time.

In this research, we will discuss some of the most important principles used to improve code quality, along with simple code examples for each principle.

We can make clean code by:

Naming Variables

"There are only two hard things in Computer Science: cache invalidation and naming things." – Phil Karlton

Variable names are important because they help developers understand the purpose of the code more easily. Good names improve readability and make the code easier to maintain.

Meaningful names provide information about the variable and help other developers understand its role without needing additional explanation.

1. Create Meaningful Names

Developers should use clear and meaningful variable names. A good name should explain the purpose of the variable without needing comments.

If a variable name requires a comment to explain it, the name should be improved instead. *"A name should tell you why it exists, what it does, and how it is used. If a name requires a comment, then the name does not reveal its intent."* – Clean Code

Bad Exp:

```
var d; // elapsed time in days
```

Good exp:

```
var elapsedTimeInDays;
```

```
var daysSinceCreation;
```

These names clearly describe what is being measured and improve code readability.

2. Avoid Disinformation

Developers should avoid using misleading names. Some words have specific meanings in programming, so they should be used correctly.

For example, the word "List" should only be used if the variable is actually a list data structure. Incorrect naming may confuse developers and lead to misunderstandings.

Bad exp:

```
var accountList = [];
```

good exp:

```
var accounts = [];
```

- **3. Avoid Noise Words**

Noise words are unnecessary words that do not add useful meaning to names. These words make code longer without improving clarity.

- Examples of noise words:
 - Info
 - Data
 - Variable
 - Object

For example, instead of using names like `UserInfo` or `BookData`, developers can simply use `User` or `Book` because the class already represents data.

Writing Functions

1. Keep Functions Small

Functions should be small and focused on a single task. Small functions are easier to read, test, understand, and maintain.

Long functions usually perform multiple responsibilities, which increases complexity and may create side effects.

A clean function should do one thing, do it well, and do only that thing. *Functions should do one thing. They should do it well. They should do it only.*

- **Encapsulate Conditionals in Functions**

Moving complex conditions into separate functions improves readability and makes the code easier to understand.

Here is a piece of code from a school project of mine. This code is responsible for inserting a chip on the board of the Connect4 game.

```
public void insertChipAt(int column) throws Exception {  
    if (isValidInsertion(column)) {  
        insertChip(column);  
        boardConfiguration += column;  
        currentPlayer = currentPlayer == Chip.RED ? Chip.YELLOW : Chip.RED;  
    } else {  
        if (!columnExistsAt(column))  
            throw new IllegalArgumentException();  
        else if (isColumnFull(column - 1) || getWinner() != Chip.NONE)
```

```

        throw new RuntimeException();
    }
}

```

The *isValidInsertion* method takes care of checking the validity of the column number and allows us the focus on the logic for inserting the chip instead.

```

private boolean isValidInsertion(int column) {

    boolean columnIsAvailable = column <= NUM_COLUMNS && column >= 1 &&
numberOfItemsInColumn[column - 1] < NUM_ROWS;

    boolean gamelsOver = getWinner() != Chip.NONE;

    return columnIsAvailable && !gamelsOver;
}

```

3. Use Fewer Arguments

Functions should have a small number of arguments whenever possible. Fewer arguments make functions easier to read, understand, and test.

3. Avoid Flag Arguments

A flag argument is a boolean argument that is passed to a function. Two different actions are taken depending on the value of this argument.

For example, say there is a function that is responsible for booking tickets to a concert and there are 2 types of users: Premium and Regular. You can have code like this:

```

public Booking book (Customer aCustomer, boolean isPremium) {

    if(isPremium)

        // logic for premium book

    else

```

```
// logic for regular booking  
}
```

Flag arguments naturally contradict the principle of single responsibility. When you see them, you should consider dividing the function into two

4. Avoid Side Effects

Side effects are unintended changes caused by a function in addition to its main task. For example, a function may unexpectedly modify a global variable or change an object passed to it.

Side effects make code harder to understand and may lead to unexpected bugs because the function performs hidden actions.

Exp:

```
let total = 0;
```

```
function addPrice(price) {  
  total += price;  
}
```

In this example, the function not only calculates the price, but also changes the value of the global variable total. This hidden behavior is considered a side effect.

File Naming

A good file name should clearly describe:

- the purpose of the file
- its responsibility
- the content inside it

Why Are Good File Names Important?

Good naming helps developers:

- read the project easily

- navigate files faster
- understand the project structure
- reduce confusion
- maintain the project more easily

Poor naming makes projects confusing and difficult to maintain.

Characteristics of Good File Names

1. Be Clear and Meaningful

The file name should explain its purpose.

Good:

login_page.dart

Bad:

file1.dart

2. Use Consistent Naming Style

Projects should follow one naming style everywhere.

Common styles:

snake_case

Words separated using underscores.

user_profile.dart

camelCase

First word lowercase, next words capitalized.

Common in javascript

PascalCase

Each word starts with a capital letter.

UserProfile.cs

Common in c# and java classes

Project Structure

Project structure in Clean Code refers to the organized arrangement of files and folders inside a software project. A well-structured project improves readability, maintainability, and scalability.

How to Create a Clean Project Structure

1. Organize Files by Responsibility

Each folder should contain files related to the same responsibility.

Example:

```
project/  
|  
├─ models/  
├─ services/  
├─ screens/  
├─ widgets/  
└─ utils/
```

- models → data classes
- services → API and business logic
- screens → application pages
- widgets → reusable UI components
- utils → helper functions

This keeps the project clean and easy to navigate.

2. Use Clear and Meaningful Names

Folders and files should describe their purpose clearly.

Good:

auth_service.dart

login_screen.dart

Bad:

data.dart

test.dart

4. Avoid Deep and Complicated Folder Structures

Too many nested folders make navigation difficult.

Bad:

project/src/main/core/utils/helpers/common/files/

Better:

project/utils/

benefits of clean project structure:

1. Easier Maintenance

Developers can quickly find and modify files.

2. Better Readability

The project becomes easier to understand.

3. Faster Development

Organized projects save development time.

4. Easier Teamwork

Team members can work together more efficiently.

Dry (Don't Repeat Yourself)

It is one of the most important principles in Clean Code and software development.

The main idea of DRY is:

Every piece of knowledge or logic should exist in only one place in the code.

Code repetition creates unnecessary complexity and increases the possibility of errors. Duplicate code means you need to change things in multiple places when there is a change in logic and it is very error prone.

Why Does Code Repetition Become a Problem?

When the same code is repeated in many places, several problems appear:

1. Harder Maintenance

If you need to change something, you must edit it everywhere.

Example:

If a tax calculation formula is repeated in 10 files, changing the formula means updating all 10 places.

This wastes time and increases the chance of mistakes.

2. More Bugs

Sometimes developers forget to update one repeated section.

As a result:

- some parts work correctly
- other parts still use old logic

This creates inconsistent behavior and bugs.

3. Bigger and Messier Code

Repeated code makes projects:

- longer

- harder to read
- difficult to understand

4. Reduced Reusability

Instead of reusing existing logic, developers may rewrite the same code multiple times, which decreases efficiency and maintainability.

Bad practice:

```
function calculateStudentTotal() {  
  let price = 100;  
  let tax = price * 0.15;  
  return price + tax;  
}
```

```
function calculateEmployeeTotal() {  
  let price = 100;  
  let tax = price * 0.15;  
  return price + tax;  
}
```

good practice:

```
function calculateTax(price) {  
  return price * 0.15;  
}
```

```
function calculateStudentTotal() {  
  let price = 100;  
  return price + calculateTax(price);  
}
```

```
function calculateEmployeeTotal() {  
  let price = 100;
```

```
return price + calculateTax(price);  
}
```

Now:

- tax logic exists in one place
- easier to maintain
- cleaner and more reusable